

1 P4 C3 — The disjoint congruence-class bound through eight

Exact cell statement. We prove the bound for every number k of classes with $2 \leq k \leq 8$. This is a complete written proof; Appendix A also supplies the complete Lean formalization and its replay instructions. We do not cite an external theorem to establish any of the seven cases.

Let $A_i = a_i \pmod{m_i}$ be k pairwise disjoint congruence classes of integers, where every m_i is a positive integer. Repeated moduli are permitted. We claim that some two of their moduli satisfy

$$\gcd(m_i, m_j) \geq k.$$

The elementary intersection criterion is

$$(a_i \pmod{m_i}) \cap (a_j \pmod{m_j}) \neq \emptyset \iff \gcd(m_i, m_j) \mid (a_j - a_i). \quad (1)$$

For the reverse implication, Bézout gives integers u, v with $g = um_i + vm_j$ for $g = \gcd(m_i, m_j)$. If $a_j - a_i = tg$, then $z = a_i + tum_i = a_j - tvm_j$ lies in both classes. The forward implication follows because an integer in both classes makes $a_j - a_i$ an integer linear combination of m_i, m_j . Thus disjointness says that a_i and a_j are *different modulo their modulus gcd*. We will use the following consequence: if $g_{ij} = \gcd(m_i, m_j)$ divides D , then $a_i \not\equiv a_j \pmod{D}$; equality modulo D would imply equality modulo g_{ij} .

Suppose, toward a contradiction for any fixed k , that every pair gcd is less than k . It cannot be 1, by (1), so throughout that case

$$2 \leq g_{ij} < k \quad (i \neq j). \quad (2)$$

1.1 The cases $k = 2, 3, 4$

For $k = 2$, the sole gcd cannot be 1, proving the claim. For $k = 3$, (2) makes every pair gcd equal to 2. Three residues would then be pairwise different modulo 2, an impossibility.

For $k = 4$, every pair gcd is 2 or 3. If all moduli are even, each pair gcd is 2, so already three residues would be distinct modulo 2. Otherwise choose an odd modulus. Its gcd with each other modulus must be 3, making every modulus divisible by 3. All pair gcds are then 3, forcing four distinct residues modulo 3. Both alternatives are impossible.

1.2 The case $k = 5$

Here every pair gcd lies in $\{2, 3, 4\}$. If all five moduli are even, each pair gcd is 2 or 4 and hence divides 4. The five residues must be pairwise distinct modulo 4, impossible.

Otherwise choose an odd modulus m_i . Its gcd with any other modulus is odd and in $\{2, 3, 4\}$, so equals 3. Consequently all five moduli are divisible by 3. Every pair gcd is now divisible by 3 and in $\{2, 3, 4\}$, so equals 3. Five pairwise distinct residues modulo 3 are impossible.

1.3 The case $k = 6$

Every pair gcd is in $\{2, 3, 4, 5\}$, so any two moduli share at least one prime from $P = \{2, 3, 5\}$. None of these three primes divides *all* six moduli. If 2 did, every pair gcd would be 2 or 4,

forcing six pairwise distinct residues modulo 4. If 3 did, every pair gcd would equal 3, forcing six distinct residues modulo 3. If 5 did, every pair gcd would equal 5, forcing six distinct residues modulo 5. Each conclusion is impossible.

Every modulus is divisible by at least two primes from P . Indeed, it is divisible by at least one because it shares a prime from P with every other modulus. If it were divisible by just one, say p , every other modulus would have to share p with it, making p common to all six.

There are only three two-prime types, $\{2, 3\}$, $\{2, 5\}$, and $\{3, 5\}$. If two moduli contained the same type, their gcd would be at least the product of its primes, hence at least 6, contradicting (2). Assign to each modulus any one of its contained two-prime types. The assignment is injective, so six moduli cannot be assigned to three types.

1.4 The case $k = 7$

Now every pair gcd is in $\{2, 3, 4, 5, 6\}$, and every pair shares at least one prime in $P = \{2, 3, 5\}$. Again no prime in P divides all seven moduli. If 3 did, each gcd would be 3 or 6 and hence divide 6; seven residues could not be pairwise distinct modulo 6. If 5 did, each gcd would equal 5; already six residues could not be pairwise distinct modulo 5.

If 2 divided all seven moduli, among the seven residues at least four would have the same parity $r \in \{0, 1\}$. Restrict to those four classes and write their residues and moduli as $a_s = 2b_s + r$ and $m_s = 2n_s$. The four reduced classes $b_s \pmod{n_s}$ are pairwise disjoint: an integer common to two reduced classes would give, after multiplying by 2 and adding r , an integer common to the corresponding original classes. The established $k = 4$ case therefore supplies a pair with $\gcd(n_s, n_t) \geq 4$. Their original gcd is $2\gcd(n_s, n_t) \geq 8$, contrary to (2). Hence 2 is not common either.

As at $k = 6$, each modulus must contain at least two primes from P : it contains one because it shares a prime from P with each other modulus, and containing only one would make that prime common to all seven. Choose an index i for which $2 \nmid m_i$ and an index j for which $3 \nmid m_j$. Then $3 \cdot 5 \mid m_i$ and $2 \cdot 5 \mid m_j$, so $i \neq j$.

Consider any of the other five indices t . If $5 \mid m_t$, it also contains 2 or 3. In the first case $10 \mid \gcd(m_t, m_j)$; in the second $15 \mid \gcd(m_t, m_i)$. Either contradicts (2). Therefore $5 \nmid m_t$, and the two-prime property forces $2 \cdot 3 \mid m_t$.

For distinct indices s, t among those five, $6 \mid \gcd(m_s, m_t)$, so (2) makes their gcd exactly 6. For each such t , the gcd of m_t and m_i is divisible by 3, is not divisible by 2 because m_i is odd, and is at most 6; it is therefore exactly 3. Their five residues must consequently be pairwise distinct modulo 6 while all avoiding $a_i \pmod{3}$. Of the six classes modulo 6, exactly four avoid one prescribed class modulo 3. Five distinct choices from four are impossible.

1.5 The case $k = 8$

Here every pair gcd belongs to $\{2, 3, 4, 5, 6, 7\}$. Remove any one index. The remaining seven classes are still disjoint, so the proved $k = 7$ case gives a pair whose gcd is at least 7. By (2), that gcd is exactly 7. Hence, for every removed index, two of the retained moduli are divisible by 7.

Let S be the set of indices whose moduli are divisible by 7. The preceding conclusion gives $|S| \geq 2$; in fact $|S| \geq 3$, because if it had exactly two members, removing either one would leave no possible pair with gcd 7. On the other hand, S is not all eight indices. If every modulus were divisible by 7, (2) would make every pair gcd equal to 7, forcing eight distinct residues modulo

7.

Choose an index $t \notin S$. For every $s \in S$, the gcd of m_s and m_t is in $\{2, 3, 4, 5, 6\}$. It is therefore divisible by one of 2, 3, 5, which divides both moduli. Distinct $s_1, s_2 \in S$ cannot share any of these three primes, because 7 and that prime would both divide $\gcd(m_{s_1}, m_{s_2})$, making it at least 14, contrary to (2). Thus each member of S requires a *different* prime from $\{2, 3, 5\}$ to intersect m_t in this divisibility sense. It follows that $|S| \leq 3$, hence $|S| = 3$.

There are therefore five indices outside S . The same argument applies to *every* outside index: its modulus is divisible by all three primes 2, 3, and 5, because it must share one distinct such prime with each of the three moduli in S . Any two outside moduli then have gcd at least $2 \cdot 3 \cdot 5 = 30$, contradicting (2). This completes the proof through eight.

The bound is sharp at each k : the k classes $0, 1, \dots, k-1 \pmod{k}$ are pairwise disjoint, and every pair of moduli has gcd exactly k .

1.6 Formal verification and source

The precise formal theorem is [ProofPursuit.P4.C3.solution](#), of type $\forall k : \text{Nat}, 2 \leq k \rightarrow k \leq 8 \rightarrow \text{ProofPursuit.P4.Statement } k$. Its Statement quantifies over arbitrary integer residues and positive natural moduli and defines disjointness using the actual integer congruence classes. The file [Progress.lean](#) closes each case, including the size-7 premise of the size-8 step. Appendix A below reproduces its complete local Lean dependency closure byte for byte.

The source package pins Lean 4.34.1 with bundled Std and no Mathlib dependency. From the repository root, run:

```
cd cagent/p4/lean
lake build ProofPursuit.P4.C3
lake env leanchecker ProofPursuit.P4.C3
```

The [verification record](#) reports both commands exiting 0 and `#print axioms` finding only Lean's standard `propext`, `Classical.choice`, and `Quot.sound`. There is no sorry, custom axiom, or native evaluation axiom in the closure. These results are reproducible checks of the formal argument, not a substitute for the written proof above.

2 Appendix A — Complete Lean proof source

The following files are reproduced in import order. Each block is the exact source file named above it; together they contain every local declaration used by `C3.solution`. The only external import is bundled Std.

2.1 ProofPursuit/P4/Basic.lean

```
import Std

namespace ProofPursuit.P4

/-- Membership in the integer congruence class of a modulo the natural modulus m. -/
def InClass (a : Int) (m : Nat) (z : Int) : Prop := (m : Int) ∣ z - a

/-- The actual set-theoretic disjointness condition, not a CRT assumption. -/
def DisjointClasses {k : Nat} (a : Fin k → Int) (m : Fin k → Nat) : Prop :=
```

```

    ∀ i j, i ≠ j → ¬ ∃ z : Int, InClass (a i) (m i) z ∧ InClass (a j) (m j) z

def Statement (k : Nat) : Prop :=
  ∀ (a : Fin k → Int) (m : Fin k → Nat),
    (∀ i, 0 < m i) → DisjointClasses a m →
      ∃ i j : Fin k, i < j ∧ k ≤ Nat.gcd (m i) (m j)

/-- Extended Euclid, proved from the standard library's gcd induction. -/
theorem bezout (m n : Nat) :
  ∃ x y : Int, (Nat.gcd m n : Int) = (m : Int) * x + (n : Int) * y := by
  induction m, n using Nat.gcd.induction with
  | H0 n => exact ⟨0, 1, by simp⟩
  | H1 m n _ ih =>
    obtain ⟨x, y, h⟩ := ih
    refine ⟨y - (n / m : Nat) * x, x, ?_⟩
    rw [Nat.gcd_rec]
    have he : (n % m : Nat) + (m : Int) * (n / m : Nat) = (n : Int) := by
      exact_mod_cast Nat.mod_add_div n m
    grind

/-- The intersection direction of the generalized Chinese remainder theorem. -/
theorem meet_of_gcd_dvd (a b : Int) (m n : Nat)
  (h : (Nat.gcd m n : Int) | b - a) :
  ∃ z : Int, InClass a m z ∧ InClass b n z := by
  obtain ⟨x, y, hb⟩ := bezout m n
  obtain ⟨t, ht⟩ := h
  refine ⟨a + (m : Int) * x * t, ?_, ?_⟩
  · exact ⟨x * t, by grind⟩
  · exact ⟨-y * t, by grind⟩

theorem incompatible {k : Nat} {a : Fin k → Int} {m : Fin k → Nat}
  (h : DisjointClasses a m) {i j : Fin k} (hij : i ≠ j) :
  ¬ (Nat.gcd (m i) (m j) : Int) | a j - a i := by
  intro hd
  exact h i j hij (meet_of_gcd_dvd _ _ _ hd)

end ProofPursuit.P4

namespace ProofPursuit.P4

theorem bounds {k : Nat} {a : Fin k → Int} {m : Fin k → Nat}
  (hm : ∀ i, 0 < m i) (hd : DisjointClasses a m)
  (hn : ¬ ∃ i j : Fin k, i < j ∧ k ≤ Nat.gcd (m i) (m j))
  {i j : Fin k} (hij : i ≠ j) :
  2 ≤ Nat.gcd (m i) (m j) ∧ Nat.gcd (m i) (m j) < k := by
  have hp := Nat.gcd_pos_of_pos_left (m j) (hm i)
  have hne : Nat.gcd (m i) (m j) ≠ 1 := by
    intro h
    have hi := incompatible hd hij
    rw [h] at hi
    exact hi (by simp)
  have hu : Nat.gcd (m i) (m j) < k := by
    apply Classical.byContradiction
    intro h
    have hl : k ≤ Nat.gcd (m i) (m j) := by omega
    by_cases hlt : i < j
    · exact hn ⟨i, j, hlt, hl⟩
    · exact hn ⟨j, i, by omega, by simp [Nat.gcd_comm] using hl⟩
  omega

theorem residues_ne {k : Nat} {a : Fin k → Int} {m : Fin k → Nat}
  (hd : DisjointClasses a m) {i j : Fin k} (hij : i ≠ j) {d : Nat}
  (hg : Nat.gcd (m i) (m j) = d) : a i % (d : Int) ≠ a j % (d : Int) := by

```

```

    intro h
    apply incompatible hd hij
    rw [hg, Int.dvd_iff_emod_eq_zero]
    exact Int.emod_eq_emod_iff_emod_sub_eq_zero.mp h.symm

end ProofPursuit.P4

namespace ProofPursuit.P4

theorem residues_ne_of_gcd_dvd {k : Nat} {a : Fin k → Int} {m : Fin k → Nat}
  (hd : DisjointClasses a m) {i j : Fin k} (hij : i ≠ j) {d : Nat}
  (hg : Nat.gcd (m i) (m j) ∣ d) : a i % (d : Int) ≠ a j % (d : Int) := by
  intro h
  apply incompatible hd hij
  have hg' : (Nat.gcd (m i) (m j) : Int) ∣ (d : Int) := by exact_mod_cast hg
  exact Int.dvd_trans hg' (Int.dvd_iff_emod_eq_zero.mpr
    (Int.emod_eq_emod_iff_emod_sub_eq_zero.mp h.symm))

end ProofPursuit.P4

```

2.2 ProofPursuit/P4/C1.lean

```

import ProofPursuit.P4.Basic

namespace ProofPursuit.P4.C1

def target : Prop := Statement 3

theorem solution : target := by
  intro a m hm hd
  apply Classical.byContradiction
  intro hn
  have hg (i j : Fin 3) (hij : i ≠ j) : Nat.gcd (m i) (m j) = 2 := by
    have := bounds hm hd hn hij
    omega
  have h01 := residues_ne hd (by decide : (0 : Fin 3) ≠ 1) (hg 0 1 (by decide))
  have h02 := residues_ne hd (by decide : (0 : Fin 3) ≠ 2) (hg 0 2 (by decide))
  have h12 := residues_ne hd (by decide : (1 : Fin 3) ≠ 2) (hg 1 2 (by decide))
  omega

end ProofPursuit.P4.C1

```

2.3 ProofPursuit/P4/C2.lean

```

import ProofPursuit.P4.Basic

namespace ProofPursuit.P4.C2

def target : Prop := Statement 4

theorem solution : target := by
  intro a m hm hd
  apply Classical.byContradiction
  intro hn
  have hb {i j : Fin 4} (hij : i ≠ j) := bounds hm hd hn hij
  by_cases he : ∀ i, 2 ∣ m i
  • have hg (i j : Fin 4) (hij : i ≠ j) : Nat.gcd (m i) (m j) = 2 := by
    have h := hb hij
    have hdiv := Nat.dvd_gcd (he i) (he j)
    have hz := Nat.mod_eq_zero_of_dvd hdiv
    omega

```

```

have h01 := residues_ne hd (by decide : (0 : Fin 4) ≠ 1) (hg 0 1 (by decide))
have h02 := residues_ne hd (by decide : (0 : Fin 4) ≠ 2) (hg 0 2 (by decide))
have h12 := residues_ne hd (by decide : (1 : Fin 4) ≠ 2) (hg 1 2 (by decide))
omega
• obtain ⟨i, hi⟩ := Classical.not_forall.mp he
have hg (j : Fin 4) (hij : i ≠ j) : Nat.gcd (m i) (m j) = 3 := by
  have h := hb hij
  have htwo : Nat.gcd (m i) (m j) ≠ 2 := by
    intro hh
    apply hi
  have hx := Nat.gcd_dvd_left (m i) (m j)
  rwa [hh] at hx
  omega
have hthree : ∀ j, 3 ∣ m j := by
  intro j
  by_cases hij : i = j
  • subst j
    have hj : ∃ j : Fin 4, i ≠ j := by
      by_cases h : i = 0
      • exact ⟨1, by omega⟩
      • exact ⟨0, h⟩
    obtain ⟨j, hj⟩ := hj
    have hx := Nat.gcd_dvd_left (m i) (m j)
    rwa [hg j hj] at hx
  • have hx := Nat.gcd_dvd_right (m i) (m j)
    rwa [hg j hij] at hx
have hall (i j : Fin 4) (hij : i ≠ j) : Nat.gcd (m i) (m j) = 3 := by
  have h := hb hij
  have hz := Nat.mod_eq_zero_of_dvd (Nat.dvd_gcd (hthree i) (hthree j))
  omega
have h01 := residues_ne hd (by decide : (0 : Fin 4) ≠ 1) (hall 0 1 (by decide))
have h02 := residues_ne hd (by decide : (0 : Fin 4) ≠ 2) (hall 0 2 (by decide))
have h03 := residues_ne hd (by decide : (0 : Fin 4) ≠ 3) (hall 0 3 (by decide))
have h12 := residues_ne hd (by decide : (1 : Fin 4) ≠ 2) (hall 1 2 (by decide))
have h13 := residues_ne hd (by decide : (1 : Fin 4) ≠ 3) (hall 1 3 (by decide))
have h23 := residues_ne hd (by decide : (2 : Fin 4) ≠ 3) (hall 2 3 (by decide))
omega
end ProofPursuit.P4.C2

```

2.4 ProofPursuit/P4/Five.lean

```

import ProofPursuit.P4.Basic

namespace ProofPursuit.P4.Five

def target : Prop := Statement 5

theorem solution : target := by
  intro a m hm hd
  apply Classical.byContradiction
  intro hn
  have hb {i j : Fin 5} (hij : i ≠ j) := bounds hm hd hn hij
  by_cases he : ∀ i, 2 ∣ m i
  • have hg (i j : Fin 5) (hij : i ≠ j) : Nat.gcd (m i) (m j) ∣ 4 := by
    have h := hb hij
    have hz := Nat.mod_eq_zero_of_dvd (Nat.dvd_gcd (he i) (he j))
    have hh : Nat.gcd (m i) (m j) = 2 ∨ Nat.gcd (m i) (m j) = 4 := by omega
    rcases hh with hh | hh <=> simp [hh]
  have h01 := residues_ne_of_gcd_dvd hd (by decide : (0 : Fin 5) ≠ 1) (hg 0 1 (by decide))
  have h02 := residues_ne_of_gcd_dvd hd (by decide : (0 : Fin 5) ≠ 2) (hg 0 2 (by decide))
  have h03 := residues_ne_of_gcd_dvd hd (by decide : (0 : Fin 5) ≠ 3) (hg 0 3 (by decide))

```

```

have h04 := residues_ne_of_gcd_dvd hd (by decide : (0 : Fin 5) ≠ 4) (hg 0 4 (by decide))
have h12 := residues_ne_of_gcd_dvd hd (by decide : (1 : Fin 5) ≠ 2) (hg 1 2 (by decide))
have h13 := residues_ne_of_gcd_dvd hd (by decide : (1 : Fin 5) ≠ 3) (hg 1 3 (by decide))
have h14 := residues_ne_of_gcd_dvd hd (by decide : (1 : Fin 5) ≠ 4) (hg 1 4 (by decide))
have h23 := residues_ne_of_gcd_dvd hd (by decide : (2 : Fin 5) ≠ 3) (hg 2 3 (by decide))
have h24 := residues_ne_of_gcd_dvd hd (by decide : (2 : Fin 5) ≠ 4) (hg 2 4 (by decide))
have h34 := residues_ne_of_gcd_dvd hd (by decide : (3 : Fin 5) ≠ 4) (hg 3 4 (by decide))
omega
• obtain ⟨i, hi⟩ := Classical.not_forall.mp he
have hg (j : Fin 5) (hij : i ≠ j) : Nat.gcd (m i) (m j) = 3 := by
  have h := hb hij
  have hn2 : ¬ 2 ∣ Nat.gcd (m i) (m j) := by
    intro hh
    exact hi (Nat.dvd_trans hh (Nat.gcd_dvd_left _ _))
  have hz : Nat.gcd (m i) (m j) % 2 ≠ 0 := by
    intro hh
    exact hn2 (Nat.dvd_of_mod_eq_zero hh)
  omega
have hthree : ∀ j, 3 ∣ m j := by
  intro j
  by_cases hij : i = j
  • subst j
    have hj : ∃ j : Fin 5, i ≠ j := by
      by_cases h : i = 0
      • exact ⟨1, by omega⟩
      • exact ⟨0, h⟩
    obtain ⟨j, hj⟩ := hj
    have hx := Nat.gcd_dvd_left (m i) (m j)
    rwa [hg j hj] at hx
  • have hx := Nat.gcd_dvd_right (m i) (m j)
    rwa [hg j hij] at hx
have hall (i j : Fin 5) (hij : i ≠ j) : Nat.gcd (m i) (m j) = 3 := by
  have h := hb hij
  have hz := Nat.mod_eq_zero_of_dvd (Nat.dvd_gcd (hthree i) (hthree j))
  omega
have h01 := residues_ne hd (by decide : (0 : Fin 5) ≠ 1) (hall 0 1 (by decide))
have h02 := residues_ne hd (by decide : (0 : Fin 5) ≠ 2) (hall 0 2 (by decide))
have h03 := residues_ne hd (by decide : (0 : Fin 5) ≠ 3) (hall 0 3 (by decide))
have h12 := residues_ne hd (by decide : (1 : Fin 5) ≠ 2) (hall 1 2 (by decide))
have h13 := residues_ne hd (by decide : (1 : Fin 5) ≠ 3) (hall 1 3 (by decide))
have h23 := residues_ne hd (by decide : (2 : Fin 5) ≠ 3) (hall 2 3 (by decide))
omega

```

end ProofPursuit.P4.Five

2.5 ProofPursuit/P4/Six.lean

```
import ProofPursuit.P4.Basic
```

```
set_option maxHeartbeats 4000000
```

```
namespace ProofPursuit.P4.Six
```

```
def target : Prop := Statement 6
```

```
theorem solution : target := by
```

```
  intro a m hm hd
```

```
  apply Classical.byContradiction
```

```
  intro hn
```

```
  have hb {i j : Fin 6} (hij : i ≠ j) := bounds hm hd hn hij
```

```
  have ncommon (p : Nat) (hp : p = 2 ∨ p = 3 ∨ p = 5) : ¬ ∀ i, p ∣ m i := by
```

```
    intro hc
```

```

have hg (i j : Fin 6) (hij : i ≠ j) := Nat.mod_eq_zero_of_dvd (Nat.dvd_gcd (hc i) (hc j))
rcases hp with rfl | rfl | rfl
· have hh (i j : Fin 6) (hij : i ≠ j) : Nat.gcd (m i) (m j) | 4 := by
  have h := hb hij
  have hz := hg i j hij
  have hx : Nat.gcd (m i) (m j) = 2 ∨ Nat.gcd (m i) (m j) = 4 := by omega
  rcases hx with hx | hx <;> simp [hx]
  have h01 := residues_ne_of_gcd_dvd hd (by decide : (0 : Fin 6) ≠ 1) (hh 0 1 (by decide))
  have h02 := residues_ne_of_gcd_dvd hd (by decide : (0 : Fin 6) ≠ 2) (hh 0 2 (by decide))
  have h03 := residues_ne_of_gcd_dvd hd (by decide : (0 : Fin 6) ≠ 3) (hh 0 3 (by decide))
  have h04 := residues_ne_of_gcd_dvd hd (by decide : (0 : Fin 6) ≠ 4) (hh 0 4 (by decide))
  have h12 := residues_ne_of_gcd_dvd hd (by decide : (1 : Fin 6) ≠ 2) (hh 1 2 (by decide))
  have h13 := residues_ne_of_gcd_dvd hd (by decide : (1 : Fin 6) ≠ 3) (hh 1 3 (by decide))
  have h14 := residues_ne_of_gcd_dvd hd (by decide : (1 : Fin 6) ≠ 4) (hh 1 4 (by decide))
  have h23 := residues_ne_of_gcd_dvd hd (by decide : (2 : Fin 6) ≠ 3) (hh 2 3 (by decide))
  have h24 := residues_ne_of_gcd_dvd hd (by decide : (2 : Fin 6) ≠ 4) (hh 2 4 (by decide))
  have h34 := residues_ne_of_gcd_dvd hd (by decide : (3 : Fin 6) ≠ 4) (hh 3 4 (by decide))
  omega
· have hh (i j : Fin 6) (hij : i ≠ j) : Nat.gcd (m i) (m j) | 3 := by
  have h := hb hij
  have hz := hg i j hij
  have hx : Nat.gcd (m i) (m j) = 3 := by omega
  simp [hx]
  have h01 := residues_ne_of_gcd_dvd hd (by decide : (0 : Fin 6) ≠ 1) (hh 0 1 (by decide))
  have h02 := residues_ne_of_gcd_dvd hd (by decide : (0 : Fin 6) ≠ 2) (hh 0 2 (by decide))
  have h03 := residues_ne_of_gcd_dvd hd (by decide : (0 : Fin 6) ≠ 3) (hh 0 3 (by decide))
  have h12 := residues_ne_of_gcd_dvd hd (by decide : (1 : Fin 6) ≠ 2) (hh 1 2 (by decide))
  have h13 := residues_ne_of_gcd_dvd hd (by decide : (1 : Fin 6) ≠ 3) (hh 1 3 (by decide))
  have h23 := residues_ne_of_gcd_dvd hd (by decide : (2 : Fin 6) ≠ 3) (hh 2 3 (by decide))
  omega
· have hh (i j : Fin 6) (hij : i ≠ j) : Nat.gcd (m i) (m j) | 5 := by
  have h := hb hij
  have hz := hg i j hij
  have hx : Nat.gcd (m i) (m j) = 5 := by omega
  simp [hx]
  have h01 := residues_ne_of_gcd_dvd hd (by decide : (0 : Fin 6) ≠ 1) (hh 0 1 (by decide))
  have h02 := residues_ne_of_gcd_dvd hd (by decide : (0 : Fin 6) ≠ 2) (hh 0 2 (by decide))
  have h03 := residues_ne_of_gcd_dvd hd (by decide : (0 : Fin 6) ≠ 3) (hh 0 3 (by decide))
  have h04 := residues_ne_of_gcd_dvd hd (by decide : (0 : Fin 6) ≠ 4) (hh 0 4 (by decide))
  have h05 := residues_ne_of_gcd_dvd hd (by decide : (0 : Fin 6) ≠ 5) (hh 0 5 (by decide))
  have h12 := residues_ne_of_gcd_dvd hd (by decide : (1 : Fin 6) ≠ 2) (hh 1 2 (by decide))
  have h13 := residues_ne_of_gcd_dvd hd (by decide : (1 : Fin 6) ≠ 3) (hh 1 3 (by decide))
  have h14 := residues_ne_of_gcd_dvd hd (by decide : (1 : Fin 6) ≠ 4) (hh 1 4 (by decide))
  have h15 := residues_ne_of_gcd_dvd hd (by decide : (1 : Fin 6) ≠ 5) (hh 1 5 (by decide))
  have h23 := residues_ne_of_gcd_dvd hd (by decide : (2 : Fin 6) ≠ 3) (hh 2 3 (by decide))
  have h24 := residues_ne_of_gcd_dvd hd (by decide : (2 : Fin 6) ≠ 4) (hh 2 4 (by decide))
  have h25 := residues_ne_of_gcd_dvd hd (by decide : (2 : Fin 6) ≠ 5) (hh 2 5 (by decide))
  have h34 := residues_ne_of_gcd_dvd hd (by decide : (3 : Fin 6) ≠ 4) (hh 3 4 (by decide))
  have h35 := residues_ne_of_gcd_dvd hd (by decide : (3 : Fin 6) ≠ 5) (hh 3 5 (by decide))
  have h45 := residues_ne_of_gcd_dvd hd (by decide : (4 : Fin 6) ≠ 5) (hh 4 5 (by decide))
  omega
have supports (i j : Fin 6) (hij : i ≠ j) :
  (2 | m i ∧ 2 | m j) ∨ (3 | m i ∧ 3 | m j) ∨ (5 | m i ∧ 5 | m j) := by
  have h := hb hij
  have hl := Nat.gcd_dvd_left (m i) (m j)
  have hr := Nat.gcd_dvd_right (m i) (m j)
  have hx : Nat.gcd (m i) (m j) = 2 ∨ Nat.gcd (m i) (m j) = 3 ∨
    Nat.gcd (m i) (m j) = 4 ∨ Nat.gcd (m i) (m j) = 5 := by omega
  rcases hx with hx | hx | hx | hx
  · exact Or.inl ⟨by simp [hx] using hl, by simp [hx] using hr⟩
  · exact Or.inr ⟨Or.inl ⟨by simp [hx] using hl, by simp [hx] using hr⟩⟩
  · exact Or.inl ⟨Nat.dvd_trans (by decide : 2 | 4) (by simp [hx] using hl),
    Nat.dvd_trans (by decide : 2 | 4) (by simp [hx] using hr)⟩

```

```

    · exact Or.inr (Or.inr (by simp [hx] using hl, by simp [hx] using hr))
have two (i : Fin 6) : (2 | m i ∧ 3 | m i) ∨ (2 | m i ∧ 5 | m i) ∨
  (3 | m i ∧ 5 | m i) := by
  have nc2 := ncommon 2 (by simp)
  have nc3 := ncommon 3 (by simp)
  have nc5 := ncommon 5 (by simp)
  by_cases h2 : 2 | m i <;> by_cases h3 : 3 | m i <;> by_cases h5 : 5 | m i
  all_goals first
    | exact Or.inl ⟨h2, h3⟩
    | exact Or.inr (Or.inl ⟨h2, h5⟩)
    | exact Or.inr (Or.inr ⟨h3, h5⟩)
    | (ex falso; apply nc2; intro j; by_cases hij : i = j
      · simp [← hij] using h2
      · rcases supports i j hij with h | h | h
        · exact h.2
        · exact False.elim (h3 h.1)
        · exact False.elim (h5 h.1))
    | (ex falso; apply nc3; intro j; by_cases hij : i = j
      · simp [← hij] using h3
      · rcases supports i j hij with h | h | h
        · exact False.elim (h2 h.1)
        · exact h.2
        · exact False.elim (h5 h.1))
    | (ex falso; apply nc5; intro j; by_cases hij : i = j
      · simp [← hij] using h5
      · rcases supports i j hij with h | h | h
        · exact False.elim (h2 h.1)
        · exact False.elim (h3 h.1)
        · exact h.2)
    | (have hx : ∃ j : Fin 6, i ≠ j := by
      by_cases h : i = 0
      · exact ⟨1, by omega⟩
      · exact ⟨0, h⟩
      obtain ⟨j, hj⟩ := hx
      rcases supports i j hj with h | h | h
      · exact False.elim (h2 h.1)
      · exact False.elim (h3 h.1)
      · exact False.elim (h5 h.1))
have clash (i j : Fin 6) (hij : i ≠ j)
  (h : (2 | m i ∧ 3 | m i ∧ 2 | m j ∧ 3 | m j) ∨
    (2 | m i ∧ 5 | m i ∧ 2 | m j ∧ 5 | m j) ∨
    (3 | m i ∧ 5 | m i ∧ 3 | m j ∧ 5 | m j)) : False := by
  have hb' := hb hij
  rcases h with h | h | h
  all_goals
    have hx := Nat.mod_eq_zero_of_dvd (Nat.dvd_gcd h.1 h.2.2.1)
    have hy := Nat.mod_eq_zero_of_dvd (Nat.dvd_gcd h.2.1 h.2.2.2)
    omega
have h0 := two 0
have h1 := two 1
have h2 := two 2
have h3 := two 3
rcases h0 with h0 | h0 | h0 <;> rcases h1 with h1 | h1 | h1 <;>
  rcases h2 with h2 | h2 | h2 <;> rcases h3 with h3 | h3 | h3
all_goals first
  | exact clash 0 1 (by decide) ((Or.inl) ⟨h0.1, h0.2, h1.1, h1.2⟩)
  | exact clash 0 1 (by decide) ((Or.inr ∘ Or.inl) ⟨h0.1, h0.2, h1.1, h1.2⟩)
  | exact clash 0 1 (by decide) ((Or.inr ∘ Or.inr) ⟨h0.1, h0.2, h1.1, h1.2⟩)
  | exact clash 0 2 (by decide) ((Or.inl) ⟨h0.1, h0.2, h2.1, h2.2⟩)
  | exact clash 0 2 (by decide) ((Or.inr ∘ Or.inl) ⟨h0.1, h0.2, h2.1, h2.2⟩)
  | exact clash 0 2 (by decide) ((Or.inr ∘ Or.inr) ⟨h0.1, h0.2, h2.1, h2.2⟩)
  | exact clash 0 3 (by decide) ((Or.inl) ⟨h0.1, h0.2, h3.1, h3.2⟩)
  | exact clash 0 3 (by decide) ((Or.inr ∘ Or.inl) ⟨h0.1, h0.2, h3.1, h3.2⟩)

```

```

| exact clash 0 3 (by decide) ((Or.inr ∘ Or.inr) ⟨h0.1, h0.2, h3.1, h3.2⟩)
| exact clash 1 2 (by decide) ((Or.inl) ⟨h1.1, h1.2, h2.1, h2.2⟩)
| exact clash 1 2 (by decide) ((Or.inr ∘ Or.inl) ⟨h1.1, h1.2, h2.1, h2.2⟩)
| exact clash 1 2 (by decide) ((Or.inr ∘ Or.inr) ⟨h1.1, h1.2, h2.1, h2.2⟩)
| exact clash 1 3 (by decide) ((Or.inl) ⟨h1.1, h1.2, h3.1, h3.2⟩)
| exact clash 1 3 (by decide) ((Or.inr ∘ Or.inl) ⟨h1.1, h1.2, h3.1, h3.2⟩)
| exact clash 1 3 (by decide) ((Or.inr ∘ Or.inr) ⟨h1.1, h1.2, h3.1, h3.2⟩)
| exact clash 2 3 (by decide) ((Or.inl) ⟨h2.1, h2.2, h3.1, h3.2⟩)
| exact clash 2 3 (by decide) ((Or.inr ∘ Or.inl) ⟨h2.1, h2.2, h3.1, h3.2⟩)
| exact clash 2 3 (by decide) ((Or.inr ∘ Or.inr) ⟨h2.1, h2.2, h3.1, h3.2⟩)

```

end ProofPursuit.P4.Six

2.6 ProofPursuit/P4/Parity.lean

import ProofPursuit.P4.C2

namespace ProofPursuit.P4

set_option maxHeartbeats 4000000

```

theorem four_same_parity (a : Fin 7 → Int) :
  ∃ i j l r : Fin 7, i < j ∧ j < l ∧ l < r ∧
    a i % 2 = a j % 2 ∧ a i % 2 = a l % 2 ∧ a i % 2 = a r % 2 := by
  by_cases h0 : a 0 % 2 = 0 <|>
    by_cases h1 : a 1 % 2 = 0 <|>
      by_cases h2 : a 2 % 2 = 0 <|>
        by_cases h3 : a 3 % 2 = 0 <|>
          by_cases h4 : a 4 % 2 = 0 <|>
            by_cases h5 : a 5 % 2 = 0 <|>
              by_cases h6 : a 6 % 2 = 0
                · exact ⟨0, 1, 2, 3, by decide, by decide, by decide, by omega, by omega, by omega⟩
                · exact ⟨0, 1, 2, 3, by decide, by decide, by decide, by omega, by omega, by omega⟩
                · exact ⟨0, 1, 2, 3, by decide, by decide, by decide, by omega, by omega, by omega⟩
                · exact ⟨0, 1, 2, 3, by decide, by decide, by decide, by omega, by omega, by omega⟩
                · exact ⟨0, 1, 2, 3, by decide, by decide, by decide, by omega, by omega, by omega⟩
                · exact ⟨0, 1, 2, 3, by decide, by decide, by decide, by omega, by omega, by omega⟩
                · exact ⟨0, 1, 2, 3, by decide, by decide, by decide, by omega, by omega, by omega⟩
                · exact ⟨0, 1, 2, 3, by decide, by decide, by decide, by omega, by omega, by omega⟩
                · exact ⟨0, 1, 2, 4, by decide, by decide, by decide, by omega, by omega, by omega⟩
                · exact ⟨0, 1, 2, 4, by decide, by decide, by decide, by omega, by omega, by omega⟩
                · exact ⟨0, 1, 2, 4, by decide, by decide, by decide, by omega, by omega, by omega⟩
                · exact ⟨0, 1, 2, 4, by decide, by decide, by decide, by omega, by omega, by omega⟩
                · exact ⟨0, 1, 2, 5, by decide, by decide, by decide, by omega, by omega, by omega⟩
                · exact ⟨0, 1, 2, 5, by decide, by decide, by decide, by omega, by omega, by omega⟩
                · exact ⟨0, 1, 2, 6, by decide, by decide, by decide, by omega, by omega, by omega⟩
                · exact ⟨3, 4, 5, 6, by decide, by decide, by decide, by omega, by omega, by omega⟩
                · exact ⟨0, 1, 3, 4, by decide, by decide, by decide, by omega, by omega, by omega⟩
                · exact ⟨0, 1, 3, 4, by decide, by decide, by decide, by omega, by omega, by omega⟩
                · exact ⟨0, 1, 3, 4, by decide, by decide, by decide, by omega, by omega, by omega⟩
                · exact ⟨0, 1, 3, 4, by decide, by decide, by decide, by omega, by omega, by omega⟩
                · exact ⟨0, 1, 3, 5, by decide, by decide, by decide, by omega, by omega, by omega⟩
                · exact ⟨0, 1, 3, 5, by decide, by decide, by decide, by omega, by omega, by omega⟩
                · exact ⟨0, 1, 3, 6, by decide, by decide, by decide, by omega, by omega, by omega⟩
                · exact ⟨2, 4, 5, 6, by decide, by decide, by decide, by omega, by omega, by omega⟩
                · exact ⟨0, 1, 4, 5, by decide, by decide, by decide, by omega, by omega, by omega⟩
                · exact ⟨0, 1, 4, 5, by decide, by decide, by decide, by omega, by omega, by omega⟩
                · exact ⟨0, 1, 4, 6, by decide, by decide, by decide, by omega, by omega, by omega⟩
                · exact ⟨0, 1, 4, 6, by decide, by decide, by decide, by omega, by omega, by omega⟩
                · exact ⟨2, 3, 5, 6, by decide, by decide, by decide, by omega, by omega, by omega⟩
                · exact ⟨0, 1, 5, 6, by decide, by decide, by decide, by omega, by omega, by omega⟩
                · exact ⟨2, 3, 4, 6, by decide, by decide, by decide, by omega, by omega, by omega⟩
                · exact ⟨2, 3, 4, 5, by decide, by decide, by decide, by omega, by omega, by omega⟩

```

[illegible]


```

apply hd (f x) (f y) (fun h => hxy (hf x y h))
refine ⟨2 * z + a i % 2, ?_, ?_⟩
· refine ⟨u, ?_⟩
  have hmx : (m (f x) : Int) = 2 * (n x : Int) := by exact_mod_cast (hmEq x).symm
  have hax := haEq x
  grind
· refine ⟨v, ?_⟩
  have hmy : (m (f y) : Int) = 2 * (n y : Int) := by exact_mod_cast (hmEq y).symm
  have hay := haEq y
  grind
obtain ⟨x, y, hxy, hg⟩ := C2.solution b n hnp hdis
have hxyn : f x ≠ f y := by intro h; have := hf x y h; omega
have hb := bounds hm hd hn hxyn
have hdiv : Nat.gcd (n x) (n y) = Nat.gcd (m (f x)) (m (f y)) / 2 :=
  Nat.gcd_div (he (f x)) (he (f y))
omega

end ProofPursuit.P4

```

2.7 ProofPursuit/P4/Seven.lean

```

import ProofPursuit.P4.Parity

namespace ProofPursuit.P4.Seven
set_option maxHeartbeats 12000000

private theorem mod3_cases (b : Int) :
  b % 3 = 0 ∨ b % 3 = 1 ∨ b % 3 = 2 := by omega

private theorem allowed0 (b x : Int) (hb : b % 3 = 0) (ha : x % 3 ≠ b % 3) :
  x % 6 ∈ ([1, 2, 4, 5] : List Int) := by simp; omega

private theorem allowed1 (b x : Int) (hb : b % 3 = 1) (ha : x % 3 ≠ b % 3) :
  x % 6 ∈ ([0, 2, 3, 5] : List Int) := by simp; omega

private theorem allowed2 (b x : Int) (hb : b % 3 = 2) (ha : x % 3 ≠ b % 3) :
  x % 6 ∈ ([0, 1, 3, 4] : List Int) := by simp; omega

private theorem mod6_range (x : Int) :
  x % 6 ∈ ([0, 1, 2, 3, 4, 5] : List Int) := by simp; omega

private theorem mod5_range (x : Int) :
  x % 5 ∈ ([0, 1, 2, 3, 4] : List Int) := by simp; omega

private theorem seven_mod6_impossible (a : Fin 7 → Int)
  (h : ∀ i j : Fin 7, i ≠ j → a i % 6 ≠ a j % 6) : False := by
  let l : List Int := [a 0 % 6, a 1 % 6, a 2 % 6, a 3 % 6,
    a 4 % 6, a 5 % 6, a 6 % 6]
  have hnd : l.Nodup := by
    simp only [l, List.nodup_cons, List.mem_cons, List.mem_nil_iff,
      or_false, not_or]
  repeat' constructor
  all_goals first
  | simp
  | exact h 0 1 (by decide)
  | exact h 0 2 (by decide)
  | exact h 0 3 (by decide)
  | exact h 0 4 (by decide)
  | exact h 0 5 (by decide)
  | exact h 0 6 (by decide)
  | exact h 1 2 (by decide)
  | exact h 1 3 (by decide)

```

```

| exact h 1 4 (by decide)
| exact h 1 5 (by decide)
| exact h 1 6 (by decide)
| exact h 2 3 (by decide)
| exact h 2 4 (by decide)
| exact h 2 5 (by decide)
| exact h 2 6 (by decide)
| exact h 3 4 (by decide)
| exact h 3 5 (by decide)
| exact h 3 6 (by decide)
| exact h 4 5 (by decide)
| exact h 4 6 (by decide)
| exact h 5 6 (by decide)
have hs : l ⊆ ([0, 1, 2, 3, 4, 5] : List Int) := by
  intro x hx
  simp [l] at hx
  rcases hx with rfl | rfl | rfl | rfl | rfl | rfl | rfl <;>
  exact mod6_range _
have := hnd.length_le_of_subset hs
simp [l] at this

private theorem six_mod5_impossible (a : Fin 7 → Int)
  (h : ∀ i j : Fin 7, i ≠ j → a i % 5 ≠ a j % 5) : False := by
  let l : List Int := [a 0 % 5, a 1 % 5, a 2 % 5,
    a 3 % 5, a 4 % 5, a 5 % 5]
  have hnd : l.Nodup := by
    simp only [l, List.nodup_cons, List.mem_cons, List.mem_nil_iff,
      or_false, not_or]
  repeat' constructor
  all_goals first
  | simp
  | exact h 0 1 (by decide)
  | exact h 0 2 (by decide)
  | exact h 0 3 (by decide)
  | exact h 0 4 (by decide)
  | exact h 0 5 (by decide)
  | exact h 1 2 (by decide)
  | exact h 1 3 (by decide)
  | exact h 1 4 (by decide)
  | exact h 1 5 (by decide)
  | exact h 2 3 (by decide)
  | exact h 2 4 (by decide)
  | exact h 2 5 (by decide)
  | exact h 3 4 (by decide)
  | exact h 3 5 (by decide)
  | exact h 4 5 (by decide)
have hs : l ⊆ ([0, 1, 2, 3, 4] : List Int) := by
  intro x hx
  simp [l] at hx
  rcases hx with rfl | rfl | rfl | rfl | rfl | rfl <;>
  exact mod5_range _
have := hnd.length_le_of_subset hs
simp [l] at this

theorem five_avoid (b x0 x1 x2 x3 x4 : Int)
  (ha0 : x0 % 3 ≠ b % 3)
  (ha1 : x1 % 3 ≠ b % 3)
  (ha2 : x2 % 3 ≠ b % 3)
  (ha3 : x3 % 3 ≠ b % 3)
  (ha4 : x4 % 3 ≠ b % 3)
  (hd01 : x0 % 6 ≠ x1 % 6)
  (hd02 : x0 % 6 ≠ x2 % 6)
  (hd03 : x0 % 6 ≠ x3 % 6)

```

```

(hd04 : x0 % 6 ≠ x4 % 6)
(hd12 : x1 % 6 ≠ x2 % 6)
(hd13 : x1 % 6 ≠ x3 % 6)
(hd14 : x1 % 6 ≠ x4 % 6)
(hd23 : x2 % 6 ≠ x3 % 6)
(hd24 : x2 % 6 ≠ x4 % 6)
(hd34 : x3 % 6 ≠ x4 % 6)
: False := by
let l : List Int := [x0 % 6, x1 % 6, x2 % 6, x3 % 6, x4 % 6]
have hnd : l.Nodup := by
  simp [l, List.nodup_cons, hd01, hd02, hd03, hd04,
    hd12, hd13, hd14, hd23, hd24, hd34]
have hb := mod3_cases b
rcases hb with hb | hb | hb
• have hs : l ⊆ ([1, 2, 4, 5] : List Int) := by
  intro y hy
  simp [l] at hy
  rcases hy with rfl | rfl | rfl | rfl | rfl
  • exact allowed0 b x0 hb ha0
  • exact allowed0 b x1 hb ha1
  • exact allowed0 b x2 hb ha2
  • exact allowed0 b x3 hb ha3
  • exact allowed0 b x4 hb ha4
  have := hnd.length_le_of_subset hs
  simp [l] at this
• have hs : l ⊆ ([0, 2, 3, 5] : List Int) := by
  intro y hy
  simp [l] at hy
  rcases hy with rfl | rfl | rfl | rfl | rfl
  • exact allowed1 b x0 hb ha0
  • exact allowed1 b x1 hb ha1
  • exact allowed1 b x2 hb ha2
  • exact allowed1 b x3 hb ha3
  • exact allowed1 b x4 hb ha4
  have := hnd.length_le_of_subset hs
  simp [l] at this
• have hs : l ⊆ ([0, 1, 3, 4] : List Int) := by
  intro y hy
  simp [l] at hy
  rcases hy with rfl | rfl | rfl | rfl | rfl
  • exact allowed2 b x0 hb ha0
  • exact allowed2 b x1 hb ha1
  • exact allowed2 b x2 hb ha2
  • exact allowed2 b x3 hb ha3
  • exact allowed2 b x4 hb ha4
  have := hnd.length_le_of_subset hs
  simp [l] at this

def target : Prop := Statement 7

theorem solution : target := by
  intro a m hm hd
  apply Classical.byContradiction
  intro hn
  have hb {i j : Fin 7} (hij : i ≠ j) := bounds hm hd hn hij
  have ncommon (p : Nat) (hp : p = 2 ∨ p = 3 ∨ p = 5) : ¬ ∀ i, p ∣ m i := by
    rcases hp with rfl | rfl | rfl
    • exact seven_not_all_even a m hm hd hn
    • intro hc
      have hh (i j : Fin 7) (hij : i ≠ j) : Nat.gcd (m i) (m j) ∣ 6 := by
        have h := hb hij
        have hz := Nat.mod_eq_zero_of_dvd (Nat.dvd_gcd (hc i) (hc j))
        have hx : Nat.gcd (m i) (m j) = 3 ∨ Nat.gcd (m i) (m j) = 6 := by omega

```

```

    rcases hx with hx | hx <|> simp [hx]
  exact seven_mod6_impossible a (fun i j hij =>
    residues_ne_of_gcd_dvd hd hij (hh i j hij))
· intro hc
  have hh (i j : Fin 7) (hij : i ≠ j) : Nat.gcd (m i) (m j) | 5 := by
    have h := hb hij
    have hz := Nat.mod_eq_zero_of_dvd (Nat.dvd_gcd (hc i) (hc j))
    have hx : Nat.gcd (m i) (m j) = 5 := by omega
    simp [hx]
  exact six_mod5_impossible a (fun i j hij =>
    residues_ne_of_gcd_dvd hd hij (hh i j hij))
have supports (i j : Fin 7) (hij : i ≠ j) :
  (2 | m i ∧ 2 | m j) ∨ (3 | m i ∧ 3 | m j) ∨ (5 | m i ∧ 5 | m j) := by
  have h := hb hij
  have hl := Nat.gcd_dvd_left (m i) (m j)
  have hr := Nat.gcd_dvd_right (m i) (m j)
  have hx : Nat.gcd (m i) (m j) = 2 ∨ Nat.gcd (m i) (m j) = 3 ∨
    Nat.gcd (m i) (m j) = 4 ∨ Nat.gcd (m i) (m j) = 5 ∨ Nat.gcd (m i) (m j) = 6 := by omega
  rcases hx with hx | hx | hx | hx | hx
  · exact Or.inl ⟨by simpa [hx] using hl, by simpa [hx] using hr⟩
  · exact Or.inr (Or.inl ⟨by simpa [hx] using hl, by simpa [hx] using hr⟩)
  · exact Or.inl ⟨Nat.dvd_trans (by decide : 2 | 4) (by simpa [hx] using hl),
    Nat.dvd_trans (by decide : 2 | 4) (by simpa [hx] using hr)⟩
  · exact Or.inr (Or.inr ⟨by simpa [hx] using hl, by simpa [hx] using hr⟩)
  · exact Or.inl ⟨Nat.dvd_trans (by decide : 2 | 6) (by simpa [hx] using hl),
    Nat.dvd_trans (by decide : 2 | 6) (by simpa [hx] using hr)⟩
have two (i : Fin 7) : (2 | m i ∧ 3 | m i) ∨ (2 | m i ∧ 5 | m i) ∨
  (3 | m i ∧ 5 | m i) := by
  have nc2 := ncommon 2 (by simp)
  have nc3 := ncommon 3 (by simp)
  have nc5 := ncommon 5 (by simp)
  by_cases h2 : 2 | m i <|> by_cases h3 : 3 | m i <|> by_cases h5 : 5 | m i
  all_goals first
  | exact Or.inl ⟨h2, h3⟩
  | exact Or.inr (Or.inl ⟨h2, h5⟩)
  | exact Or.inr (Or.inr ⟨h3, h5⟩)
  | (ex falso; apply nc2; intro j; by_cases hij : i = j
    · simpa [← hij] using h2
    · rcases supports i j hij with h | h | h
      · exact h.2
      · exact False.elim (h3 h.1)
      · exact False.elim (h5 h.1))
  | (ex falso; apply nc3; intro j; by_cases hij : i = j
    · simpa [← hij] using h3
    · rcases supports i j hij with h | h | h
      · exact False.elim (h2 h.1)
      · exact h.2
      · exact False.elim (h5 h.1))
  | (ex falso; apply nc5; intro j; by_cases hij : i = j
    · simpa [← hij] using h5
    · rcases supports i j hij with h | h | h
      · exact False.elim (h2 h.1)
      · exact False.elim (h3 h.1)
      · exact h.2)
  | (have hx : ∃ j : Fin 7, i ≠ j := by
    by_cases h : i = 0
    · exact ⟨1, by omega⟩
    · exact ⟨0, h⟩
    obtain ⟨j, hj⟩ := hx
    rcases supports i j hj with h | h | h
    · exact False.elim (h2 h.1)
    · exact False.elim (h3 h.1)
    · exact False.elim (h5 h.1))

```

```

obtain ⟨i, hi⟩ := Classical.not_forall.mp (ncommon 2 (by simp))
obtain ⟨j, hj⟩ := Classical.not_forall.mp (ncommon 3 (by simp))
have hi35 : 3 ∣ m i ∧ 5 ∣ m i := by
  rcases two i with h | h | h
  · exact False.elim (hi h.1)
  · exact False.elim (hi h.1)
  · exact h
have hj25 : 2 ∣ m j ∧ 5 ∣ m j := by
  rcases two j with h | h | h
  · exact False.elim (hj h.2)
  · exact h
  · exact False.elim (hj h.1)
have ht23 (t : Fin 7) (hti : t ≠ i) (htj : t ≠ j) : 2 ∣ m t ∧ 3 ∣ m t := by
  have h5 : ¬ 5 ∣ m t := by
    intro h5
    rcases two t with h | h | h
    · have b := hb htj
      have x := Nat.mod_eq_zero_of_dvd (Nat.dvd_gcd h.1 hj25.1)
      have y := Nat.mod_eq_zero_of_dvd (Nat.dvd_gcd h5 hj25.2)
      omega
    · have b := hb htj
      have x := Nat.mod_eq_zero_of_dvd (Nat.dvd_gcd h.1 hj25.1)
      have y := Nat.mod_eq_zero_of_dvd (Nat.dvd_gcd h5 hj25.2)
      omega
    · have b := hb hti
      have x := Nat.mod_eq_zero_of_dvd (Nat.dvd_gcd h.1 hi35.1)
      have y := Nat.mod_eq_zero_of_dvd (Nat.dvd_gcd h5 hi35.2)
      omega
  rcases two t with h | h | h
  · exact h
  · exact False.elim (h5 h.2)
  · exact False.elim (h5 h.2)
have avoid (t : Fin 7) (hti : t ≠ i) (htj : t ≠ j) : a t % 3 ≠ a i % 3 := by
  have ht := ht23 t hti htj
  have b := hb hti
  have h3 := Nat.mod_eq_zero_of_dvd (Nat.dvd_gcd ht.2 hi35.1)
  have h6 : Nat.gcd (m t) (m i) ≠ 6 := by
    intro h
    apply hi
    exact Nat.dvd_trans (by decide : 2 ∣ 6) (by simpa [h] using Nat.gcd_dvd_right (m t) (m i))
  have hg : Nat.gcd (m t) (m i) = 3 := by omega
  exact residues_ne hd hti hg
have different (t u : Fin 7) (htu : t ≠ u) (hti : t ≠ i) (htj : t ≠ j)
  (hui : u ≠ i) (huj : u ≠ j) : a t % 6 ≠ a u % 6 := by
  have ht := ht23 t hti htj
  have hu := ht23 u hui huj
  have b := hb htu
  have h2 := Nat.mod_eq_zero_of_dvd (Nat.dvd_gcd ht.1 hu.1)
  have h3 := Nat.mod_eq_zero_of_dvd (Nat.dvd_gcd ht.2 hu.2)
  have hg : Nat.gcd (m t) (m u) = 6 := by omega
  exact residues_ne hd htu hg
have hic : i = 0 ∨ i = 1 ∨ i = 2 ∨ i = 3 ∨ i = 4 ∨ i = 5 ∨ i = 6 := by omega
have hjc : j = 0 ∨ j = 1 ∨ j = 2 ∨ j = 3 ∨ j = 4 ∨ j = 5 ∨ j = 6 := by omega
rcases hic with rfl | rfl | rfl | rfl | rfl | rfl | rfl <|>
  rcases hjc with rfl | rfl | rfl | rfl | rfl | rfl | rfl
· exact hi hj25.1
· exact five_avoid (a 0) (a 2) (a 3) (a 4) (a 5) (a 6)
  (avoid 2 (by decide) (by decide))
  (avoid 3 (by decide) (by decide))
  (avoid 4 (by decide) (by decide))
  (avoid 5 (by decide) (by decide))
  (avoid 6 (by decide) (by decide))
  (different 2 3 (by decide) (by decide) (by decide) (by decide) (by decide))

```


[illegible]

[illegible]


```

    (different 0 4 (by decide) (by decide) (by decide) (by decide) (by decide))
    (different 0 5 (by decide) (by decide) (by decide) (by decide) (by decide))
    (different 1 2 (by decide) (by decide) (by decide) (by decide) (by decide))
    (different 1 4 (by decide) (by decide) (by decide) (by decide) (by decide))
    (different 1 5 (by decide) (by decide) (by decide) (by decide) (by decide))
    (different 2 4 (by decide) (by decide) (by decide) (by decide) (by decide))
    (different 2 5 (by decide) (by decide) (by decide) (by decide) (by decide))
    (different 4 5 (by decide) (by decide) (by decide) (by decide) (by decide))
  • exact five_avoid (a 6) (a 0) (a 1) (a 2) (a 3) (a 5)
    (avoid 0 (by decide) (by decide))
    (avoid 1 (by decide) (by decide))
    (avoid 2 (by decide) (by decide))
    (avoid 3 (by decide) (by decide))
    (avoid 5 (by decide) (by decide))
    (different 0 1 (by decide) (by decide) (by decide) (by decide) (by decide))
    (different 0 2 (by decide) (by decide) (by decide) (by decide) (by decide))
    (different 0 3 (by decide) (by decide) (by decide) (by decide) (by decide))
    (different 0 5 (by decide) (by decide) (by decide) (by decide) (by decide))
    (different 1 2 (by decide) (by decide) (by decide) (by decide) (by decide))
    (different 1 3 (by decide) (by decide) (by decide) (by decide) (by decide))
    (different 1 5 (by decide) (by decide) (by decide) (by decide) (by decide))
    (different 2 3 (by decide) (by decide) (by decide) (by decide) (by decide))
    (different 2 5 (by decide) (by decide) (by decide) (by decide) (by decide))
    (different 3 5 (by decide) (by decide) (by decide) (by decide) (by decide))
  • exact five_avoid (a 6) (a 0) (a 1) (a 2) (a 3) (a 4)
    (avoid 0 (by decide) (by decide))
    (avoid 1 (by decide) (by decide))
    (avoid 2 (by decide) (by decide))
    (avoid 3 (by decide) (by decide))
    (avoid 4 (by decide) (by decide))
    (different 0 1 (by decide) (by decide) (by decide) (by decide) (by decide))
    (different 0 2 (by decide) (by decide) (by decide) (by decide) (by decide))
    (different 0 3 (by decide) (by decide) (by decide) (by decide) (by decide))
    (different 0 4 (by decide) (by decide) (by decide) (by decide) (by decide))
    (different 1 2 (by decide) (by decide) (by decide) (by decide) (by decide))
    (different 1 3 (by decide) (by decide) (by decide) (by decide) (by decide))
    (different 1 4 (by decide) (by decide) (by decide) (by decide) (by decide))
    (different 2 3 (by decide) (by decide) (by decide) (by decide) (by decide))
    (different 2 4 (by decide) (by decide) (by decide) (by decide) (by decide))
    (different 3 4 (by decide) (by decide) (by decide) (by decide) (by decide))
  • exact hi hj25.1

```

end ProofPursuit.P4.Seven

2.8 ProofPursuit/P4/Eight.lean

```
import ProofPursuit.P4.Basic
```

```
namespace ProofPursuit.P4.Eight
```

```
set_option maxHeartbeats 12000000
```

```
private def omitIndex (t : Fin 8) (s : Fin 7) : Fin 8 :=
  {if s.val < t.val then s.val else s.val + 1, by
   by_cases h : s.val < t.val <=> simp [h] <=> omega}
```

```
private theorem omitIndex_ne (t : Fin 8) (s : Fin 7) : omitIndex t s ≠ t := by
  intro h
  have hv := congrArg Fin.val h
  simp only [omitIndex] at hv
  by_cases hs : s.val < t.val <=> simp [hs] at hv <=> omega

```

```

private theorem omitIndex_injective (t : Fin 8) {s u : Fin 7}
  (h : omitIndex t s = omitIndex t u) : s = u := by
  have hv := congrArg Fin.val h
  simp only [omitIndex] at hv
  by_cases hs : s.val < t.val <;> by_cases hu : u.val < t.val <;>
  simp [hs, hu] at hv <;> apply Fin.ext <;> omega

private theorem omitIndex_surjective (t i : Fin 8) (hit : i ≠ t) :
  ∃ s : Fin 7, omitIndex t s = i := by
  have hv : i.val ≠ t.val := by intro h; exact hit (Fin.ext h)
  by_cases h : i.val < t.val
  · refine ⟨⟨i.val, by omega⟩, ?_⟩
    apply Fin.ext
    simp [omitIndex, h]
  · refine ⟨⟨i.val - 1, by omega⟩, ?_⟩
    apply Fin.ext
    simp [omitIndex, show ¬ i.val - 1 < t.val by omega]
    omega

def target : Prop := Statement 8

theorem step (h7 : Statement 7) : target := by
  intro a m hm hd
  apply Classical.byContradiction
  intro hn
  have hb {i j : Fin 8} (hij : i ≠ j) := bounds hm hd hn hij
  have sevenPair (t : Fin 8) :
    ∃ i j : Fin 8, i ≠ j ∧ i ≠ t ∧ j ≠ t ∧ Nat.gcd (m i) (m j) = 7 := by
    let a' : Fin 7 → Int := fun s => a (omitIndex t s)
    let m' : Fin 7 → Nat := fun s => m (omitIndex t s)
    have hm' : ∀ s, 0 < m' s := by intro s; exact hm _
    have hd' : DisjointClasses a' m' := by
      intro s u hsu
      exact hd (omitIndex t s) (omitIndex t u)
      (fun h => hsu (omitIndex_injective t h))
    obtain ⟨s, u, hsu, hge⟩ := h7 a' m' hm' hd'
    have hne : omitIndex t s ≠ omitIndex t u := by
      intro h
      have := omitIndex_injective t h
      omega
    refine ⟨omitIndex t s, omitIndex t u, hne, omitIndex_ne t s,
      omitIndex_ne t u, ?_⟩
    have hlt := (hb hne).2
    dsimp [m'] at hge
    omega
  have gcdSeven (x y : Fin 8) (hxy : x ≠ y)
    (hx : 7 ∣ m x) (hy : 7 ∣ m y) : Nat.gcd (m x) (m y) = 7 := by
    have h := hb hxy
    have hz := Nat.mod_eq_zero_of_dvd (Nat.dvd_gcd hx hy)
    omega
  have noAllSeven : ¬ ∀ x : Fin 8, 7 ∣ m x := by
    intro hall
    have hres (x y : Fin 8) (hxy : x ≠ y) : a x % 7 ≠ a y % 7 :=
      residues_ne hd hxy (gcdSeven x y hxy (hall x) (hall y))
    let l : List Int := List.ofFn (fun x : Fin 8 => a x % 7)
    have hnodup : l.Nodup := by
      rw [List.nodup_iff_eq_of_getElem_eq]
      intro x y hx hy heq
      have hx8 : x < 8 := by simp [l] using hx
      have hy8 : y < 8 := by simp [l] using hy
      apply Classical.byContradiction
      intro hxy
      have hfi : ⟨x, hx8⟩ : Fin 8 ≠ ⟨y, hy8⟩ := by

```

```

    intro h
    apply hxy
    exact congrArg Fin.val h
    have hne := hres ⟨x, hx8⟩ ⟨y, hy8⟩ hfi
    apply hne
    simp only [l, List.getElem_ofFn] using heq
    have hsub : l ⊆ ([0, 1, 2, 3, 4, 5, 6] : List Int) := by
      intro x hx
      obtain ⟨i, rfl⟩ := List.mem_ofFn.mp hx
      have h0 := Int.emod_nonneg (a i) (by decide : (7 : Int) ≠ 0)
      have h1 := Int.emod_lt_of_pos (a i) (by decide : (0 : Int) < 7)
      simp only [List.mem_cons, List.mem_nil_iff, or_false]
      omega
    have hc := hnodup.length_le_of_subset hsub
    simp [l] at hc
    obtain ⟨i, j, hij, _, _, hij7⟩ := sevenPair 0
    have hi7 : 7 ∣ m i := by
      rw [← hij7]
      exact Nat.gcd_dvd_left (m i) (m j)
    have hj7 : 7 ∣ m j := by
      rw [← hij7]
      exact Nat.gcd_dvd_right (m i) (m j)
    have third : ∃ k : Fin 8, k ≠ i ∧ k ≠ j ∧ 7 ∣ m k := by
      obtain ⟨p, q, hpq, hpi, hqi, hpq7⟩ := sevenPair i
      by_cases hpj : p = j
      · refine ⟨q, hqi, ?_, ?_⟩
        · exact fun h => hpq (hpj.trans h.symm)
        · rw [← hpq7]
          exact Nat.gcd_dvd_right (m p) (m q)
      · refine ⟨p, hpi, hpj, ?_⟩
        rw [← hpq7]
        exact Nat.gcd_dvd_left (m p) (m q)
    obtain ⟨k, hki, hkj, hk7⟩ := third
    obtain ⟨t, ht7⟩ := Classical.not_forall.mp noAllSeven
    have hti : t ≠ i := by intro h; exact ht7 (h ▸ hi7)
    have htj : t ≠ j := by intro h; exact ht7 (h ▸ hj7)
    have htk : t ≠ k := by intro h; exact ht7 (h ▸ hk7)
    have noCommon (x y : Fin 8) (hxy : x ≠ y) :
      (hx : 7 ∣ m x) (hy : 7 ∣ m y) (p : Nat)
      (hp : p = 2 ∨ p = 3 ∨ p = 5) : ¬(p ∣ m x ∧ p ∣ m y) := by
      intro h
      have hdvd : p ∣ 7 := by
        rw [← gcdSeven x y hxy hx hy]
        exact Nat.dvd_gcd h.1 h.2
      rcases hp with rfl | rfl | rfl
      · exact (by decide : ¬ 2 ∣ 7) hdvd
      · exact (by decide : ¬ 3 ∣ 7) hdvd
      · exact (by decide : ¬ 5 ∣ 7) hdvd
    have sharedSmall (x y : Fin 8) (hxy : x ≠ y) (hy : ¬ 7 ∣ m y) :
      (2 ∣ m x ∧ 2 ∣ m y) ∨
      (3 ∣ m x ∧ 3 ∣ m y) ∨
      (5 ∣ m x ∧ 5 ∣ m y) := by
      have h := hb hxy
      have hl := Nat.gcd_dvd_left (m x) (m y)
      have hr := Nat.gcd_dvd_right (m x) (m y)
      have hne : Nat.gcd (m x) (m y) ≠ 7 := by
        intro heq
        apply hy
        rw [← heq]
        exact hr
      have hx : Nat.gcd (m x) (m y) = 2 ∨ Nat.gcd (m x) (m y) = 3 ∨
        Nat.gcd (m x) (m y) = 4 ∨ Nat.gcd (m x) (m y) = 5 ∨
        Nat.gcd (m x) (m y) = 6 := by omega

```

```

rcases hx with hx | hx | hx | hx | hx
• exact Or.inl ⟨by simp [hx] using hl, by simp [hx] using hr⟩
• exact Or.inr (Or.inl ⟨by simp [hx] using hl, by simp [hx] using hr⟩)
• exact Or.inl ⟨Nat.dvd_trans (by decide : 2 | 4) (by simp [hx] using hl),
  Nat.dvd_trans (by decide : 2 | 4) (by simp [hx] using hr)⟩
• exact Or.inr (Or.inr ⟨by simp [hx] using hl, by simp [hx] using hr⟩)
• exact Or.inl ⟨Nat.dvd_trans (by decide : 2 | 6) (by simp [hx] using hl),
  Nat.dvd_trans (by decide : 2 | 6) (by simp [hx] using hr)⟩
have h_it := sharedSmall i t hti.symm ht7
have h_jt := sharedSmall j t htj.symm ht7
have h_kt := sharedSmall k t htk.symm ht7
have h_ij2 := noCommon i j hij hi7 hj7 2 (Or.inl rfl)
have h_ij3 := noCommon i j hij hi7 hj7 3 (Or.inr (Or.inl rfl))
have h_ij5 := noCommon i j hij hi7 hj7 5 (Or.inr (Or.inr rfl))
have h_ik2 := noCommon i k hki.symm hi7 hk7 2 (Or.inl rfl)
have h_ik3 := noCommon i k hki.symm hi7 hk7 3 (Or.inr (Or.inl rfl))
have h_ik5 := noCommon i k hki.symm hi7 hk7 5 (Or.inr (Or.inr rfl))
have h_jk2 := noCommon j k hkj.symm hj7 hk7 2 (Or.inl rfl)
have h_jk3 := noCommon j k hkj.symm hj7 hk7 3 (Or.inr (Or.inl rfl))
have h_jk5 := noCommon j k hkj.symm hj7 hk7 5 (Or.inr (Or.inr rfl))
have ht235 : 2 | m t ^ 3 | m t ^ 5 | m t := by grind
have fifth : ∃ u : Fin 8, u ≠ i ∧ u ≠ j ∧ u ≠ k ∧ u ≠ t := by
  apply Classical.byContradiction
  intro h
  have h0 : ¬ ((0 : Fin 8) ≠ i ∧ 0 ≠ j ∧ 0 ≠ k ∧ 0 ≠ t) := by
    intro hh; exact h ⟨0, hh⟩
  have h1 : ¬ ((1 : Fin 8) ≠ i ∧ 1 ≠ j ∧ 1 ≠ k ∧ 1 ≠ t) := by
    intro hh; exact h ⟨1, hh⟩
  have h2 : ¬ ((2 : Fin 8) ≠ i ∧ 2 ≠ j ∧ 2 ≠ k ∧ 2 ≠ t) := by
    intro hh; exact h ⟨2, hh⟩
  have h3 : ¬ ((3 : Fin 8) ≠ i ∧ 3 ≠ j ∧ 3 ≠ k ∧ 3 ≠ t) := by
    intro hh; exact h ⟨3, hh⟩
  have h4 : ¬ ((4 : Fin 8) ≠ i ∧ 4 ≠ j ∧ 4 ≠ k ∧ 4 ≠ t) := by
    intro hh; exact h ⟨4, hh⟩
  omega
obtain ⟨u, hui, huj, huk, hut⟩ := fifth
have hu7 : ¬ 7 | m u := by
  intro hu7
  have h_ut := sharedSmall u t hut ht7
  have h_uui2 := noCommon u i hui hu7 hi7 2 (Or.inl rfl)
  have h_uui3 := noCommon u i hui hu7 hi7 3 (Or.inr (Or.inl rfl))
  have h_uui5 := noCommon u i hui hu7 hi7 5 (Or.inr (Or.inr rfl))
  have h_uuj2 := noCommon u j huj hu7 hj7 2 (Or.inl rfl)
  have h_uuj3 := noCommon u j huj hu7 hj7 3 (Or.inr (Or.inl rfl))
  have h_uuj5 := noCommon u j huj hu7 hj7 5 (Or.inr (Or.inr rfl))
  have h_uk2 := noCommon u k huk hu7 hk7 2 (Or.inl rfl)
  have h_uk3 := noCommon u k huk hu7 hk7 3 (Or.inr (Or.inl rfl))
  have h_uk5 := noCommon u k huk hu7 hk7 5 (Or.inr (Or.inr rfl))
  grind
have h_iu := sharedSmall i u hui.symm hu7
have h_ju := sharedSmall j u huj.symm hu7
have h_ku := sharedSmall k u huk.symm hu7
have hu235 : 2 | m u ^ 3 | m u ^ 5 | m u := by grind
have hbound := hb hut.symm
have hg2 := Nat.mod_eq_zero_of_dvd (Nat.dvd_gcd ht235.1 hu235.1)
have hg3 := Nat.mod_eq_zero_of_dvd (Nat.dvd_gcd ht235.2.1 hu235.2.1)
have hg5 := Nat.mod_eq_zero_of_dvd (Nat.dvd_gcd ht235.2.2 hu235.2.2)
omega

```

end ProofPursuit.P4.Eight

2.9 ProofPursuit/P4/Progress.lean

```
import ProofPursuit.P4.C1
import ProofPursuit.P4.C2
import ProofPursuit.P4.Five
import ProofPursuit.P4.Six
import ProofPursuit.P4.Seven
import ProofPursuit.P4.Eight

namespace ProofPursuit.P4.Progress

/-- The k = 2 base case, included to make the certified range contiguous. -/
theorem two : Statement 2 := by
  intro a m hm hd
  apply Classical.byContradiction
  intro hn
  have := bounds hm hd hn (by decide : (0 : Fin 2) ≠ 1)
  omega

/-- The complete contiguous range required by P4 C3. -/
def target : Prop := ∀ k : Nat, 2 ≤ k → k ≤ 8 → Statement k

theorem solution : target := by
  intro k hlo hhi
  have hk : k = 2 ∨ k = 3 ∨ k = 4 ∨ k = 5 ∨ k = 6 ∨ k = 7 ∨ k = 8 := by omega
  rcases hk with rfl | rfl | rfl | rfl | rfl | rfl | rfl
  · exact two
  · exact C1.solution
  · exact C2.solution
  · exact Five.solution
  · exact Six.solution
  · exact Seven.solution
  · exact Eight.step Seven.solution

#print axioms solution
#print axioms C1.solution
#print axioms C2.solution
#print axioms Five.solution
#print axioms Six.solution
#print axioms Seven.solution
#print axioms Eight.step

end ProofPursuit.P4.Progress
```

2.10 ProofPursuit/P4/C3.lean

```
import ProofPursuit.P4.Progress

namespace ProofPursuit.P4.C3

/-- Every size from 2 through 8, with no assumption about smaller cases. -/
def target : Prop := ∀ k : Nat, 2 ≤ k → k ≤ 8 → Statement k

theorem solution : target := Progress.solution

#print axioms solution

end ProofPursuit.P4.C3
```